Full Stack AI Security

by Jonathan Marcil
February 2026

Content overview

- **Threat Modeling**
  The big picture of securing AI
- **Data Handling**
  Ties threat model with actions to take during data lifecycle
- **Security Actions**
  A selection of practical actions based on threats against our most important data

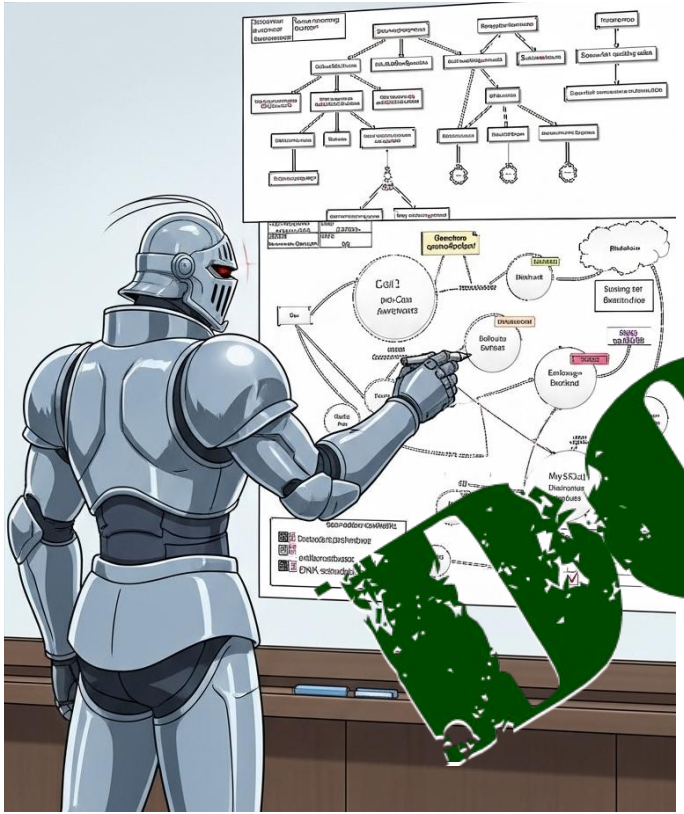
What is Threat Modeling?

We recognize what can go wrong…

To pinpoint security issues and weaknesses…

That we use to inform decisions on needed actions…

How to.. Threat Model?



List attacker goals

Draw attack surface

Select what to secure

DONE

Attacker Goals

Gain access to resources

Steal private individual information (PII)

Steal intellectual property (your efforts)

What attackers want really?

Make money

→ Resell information or credentials

Save money

→ Use your AI credits for their own usage

flare

Report

Thousands Secrets Fo Docker Hu Putting Or at Risk

Key Findings

More than **10,000** Docker Hub images contained leaked secrets (including live credentials to production systems) found in just one month of scanning

AI **LLM model keys were the most frequently leaked credentials**, with almost **4,000 exposed**, revealing how fast AI adoption has outpaced security controls

Attack Surface

What can go wrong with your systems

+

What is exposed on the Internet

=

What is probable to be targeted

What to secure?

=

What is probable to be targeted

+

Possible attacks

but knowing about attacks and threats requires lots and lots of knowledge...

What can we use as a tool in 2026 for that...?

LLM powered analysis

If you already have a code base…

Security threat model for OpenClaw

openclaw/openclaw

can you help me do a security threat model and list out:
What can go wrong with this system
What is exposed on the Internet
What is probable to be targeted

Copilot finished thinking

I'll help you create a security threat model for the openclaw/openclaw repository. Let me first gather information about the repository's structure, dependencies, and what it does.Based on my analysis of the OpenClaw repository, here's a comprehensive security threat model:

LLM example: OPENCLAW

OpenClaw is a personal AI assistant you run on your own devices.

It answers you on the channels you already use (WhatsApp, Telegram, Slack, Discord, Google Chat, Signal, iMessage, Microsoft Teams, WebChat), plus extension channels like BlueBubbles, Matrix, Zalo, and Zalo Personal. It can speak and listen on macOS/iOS/Android, and can render a live Canvas you control. The Gateway is just the control plane — the product is the assistant.

LLM example prompt

do a security threat model and list out:
What can go wrong with this system,
What is exposed on the Internet,
What is probable to be targeted

(Copilot Claude Haiku 4.5, within GitHub Repo)

What can go wrong

1. Agent Execution Risks
2. Authentication & Authorization
3. Data Exposure
4. Network & Communication
5. Plugin & Extension Risks
6. Memory & Context Management
7. Access Control Issues
8. Supply Chain & Dependencies

What can go wrong

1. Agent Execution Risks
   - Arbitrary command execution on local machines (by design, but needs careful controls)
   - Unintended file system access or modification
2. Authentication & Authorization
   - *Weak* or compromised API tokens (Discord, Gmail, BlueBubbles, GitHub, etc.)

What can go wrong

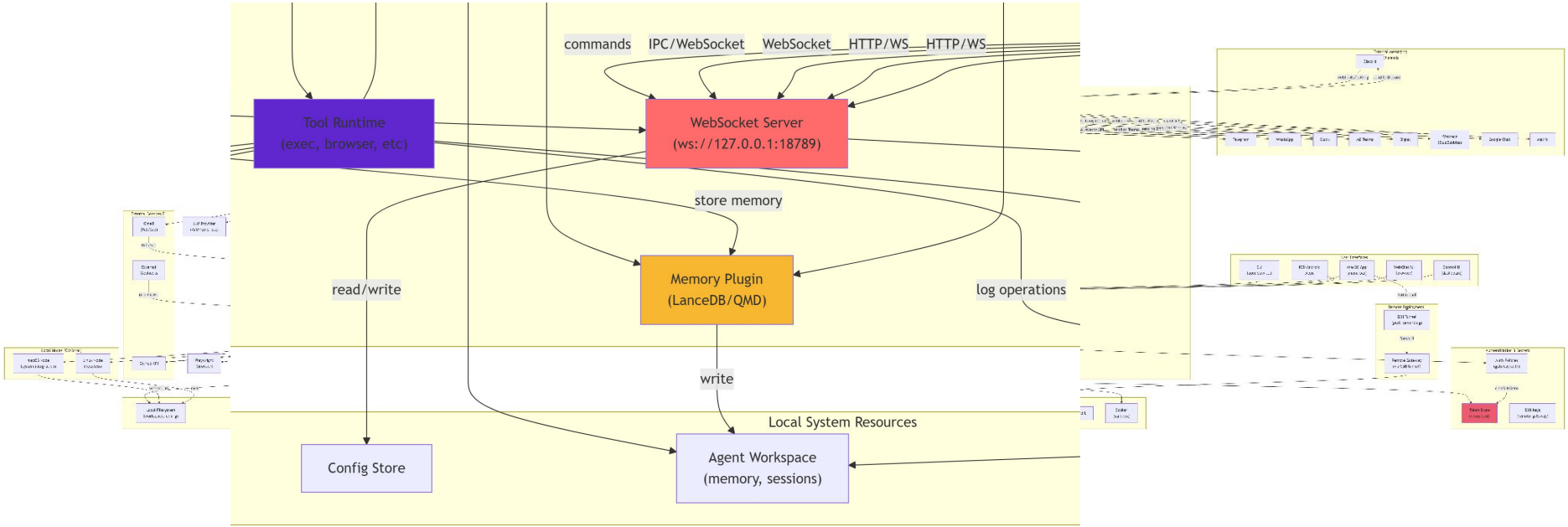
Data Exposure

- Unencrypted storage of **sensitive credentials in config files**
- SSH tunnel credentials exposure
- Message content logging without sanitization
- Memory/context storage containing PII without proper controls
- Webhook payloads containing sensitive data logged

What Is Exposed on the Internet

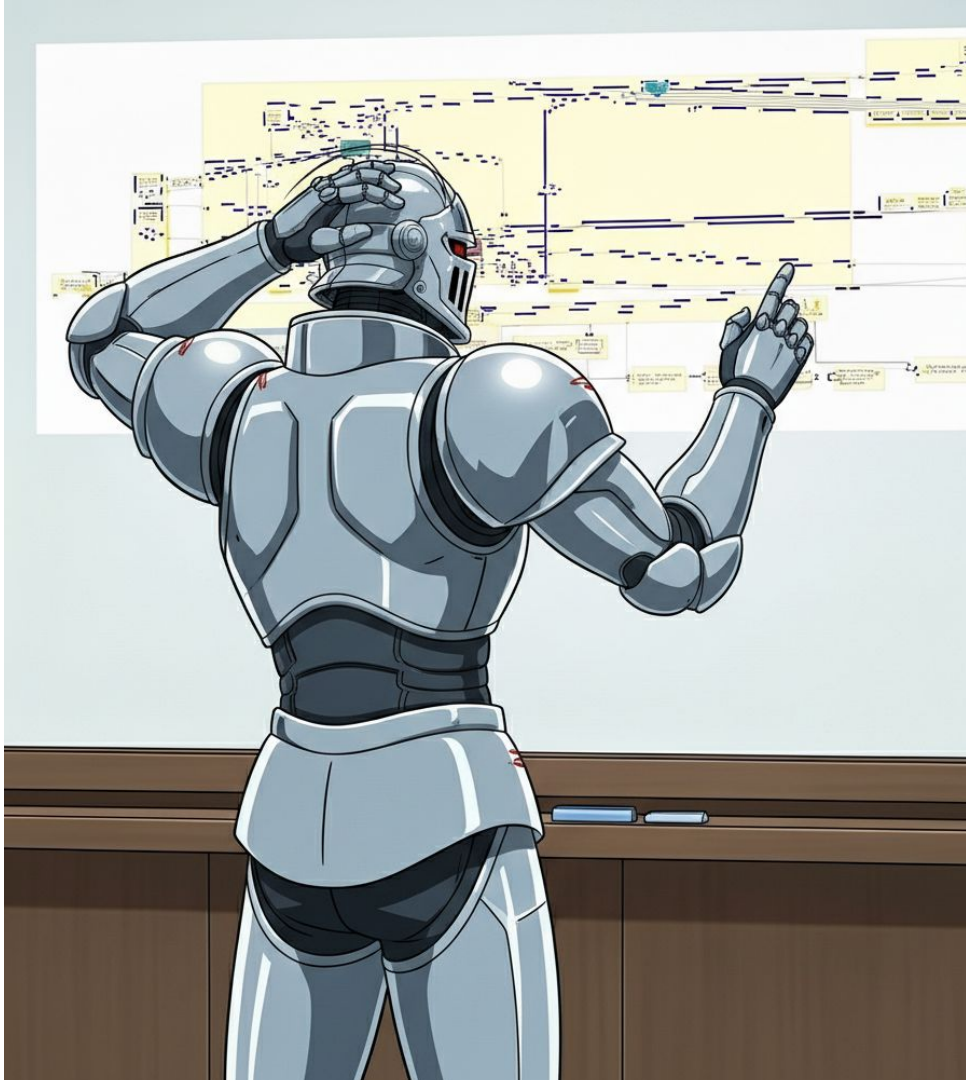
1. External Integration Points
2. Remote Gateway Surface
3. Control UI/Web Surface
4. API Endpoints

LLM: create a data flow diagram



Warning: LLM for Threat Modeling

More often than not, the LLM will **OVERDELIVER** what you have asked them, use it as a brainstorming to get started but organize ideas yourself!



LLM OpenClaw: List data involved

🔴 CRITICAL (Compromise = Full System Takeover)

- API Tokens (Discord, GitHub, Gmail, etc.)
- Gateway Auth Token
- LLM Provider Keys
- SSH Private Keys
- OAuth Refresh Tokens
- WhatsApp/Signal Credentials

🟠 HIGH (Compromise = Privacy Breach + Impersonation)

- Session Transcripts (full conversation history)
- Semantic Memory Database
- User Phone Numbers (WhatsApp, Signal)
- Discord/Slack User IDs (allowlists)
- Private Messages & DMs
- Configuration with tokens

From data flow to attack vector

Data flow tells where data goes +

The attacker goals identify data they want +

Attack surface tells you where they might enter

=

Likely attack vectors

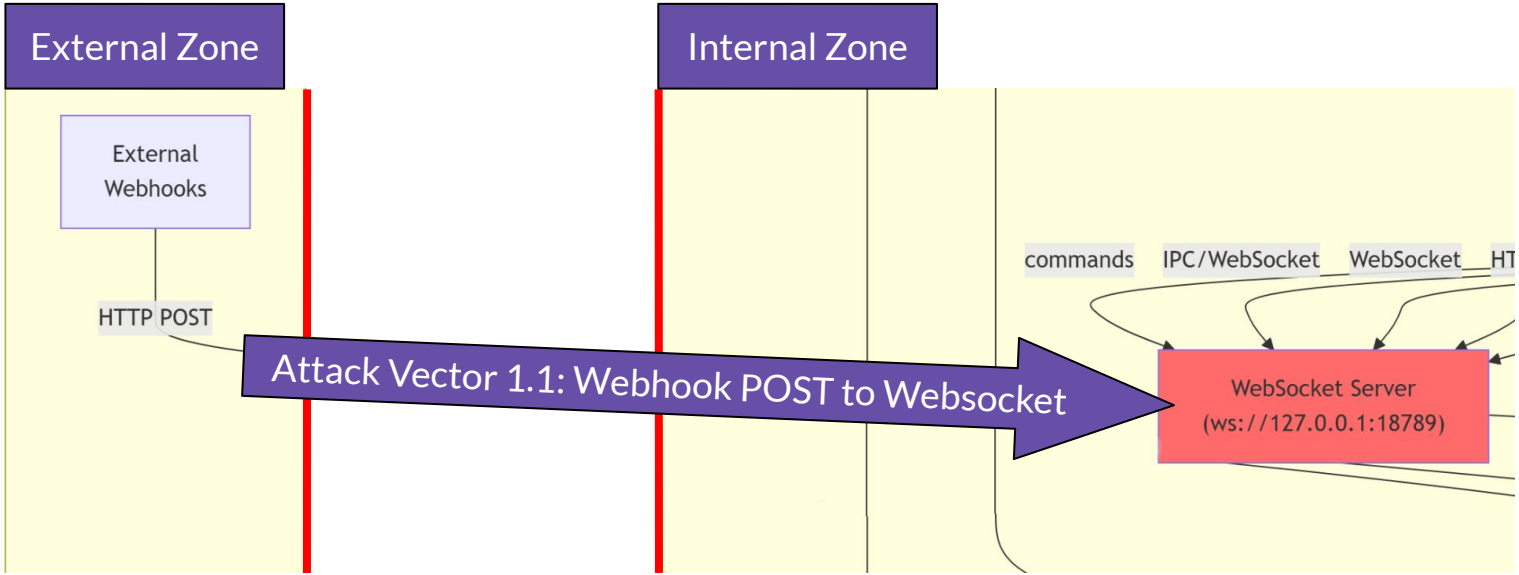
Attack vectors

Starts at entry point and traverse one zone to another.

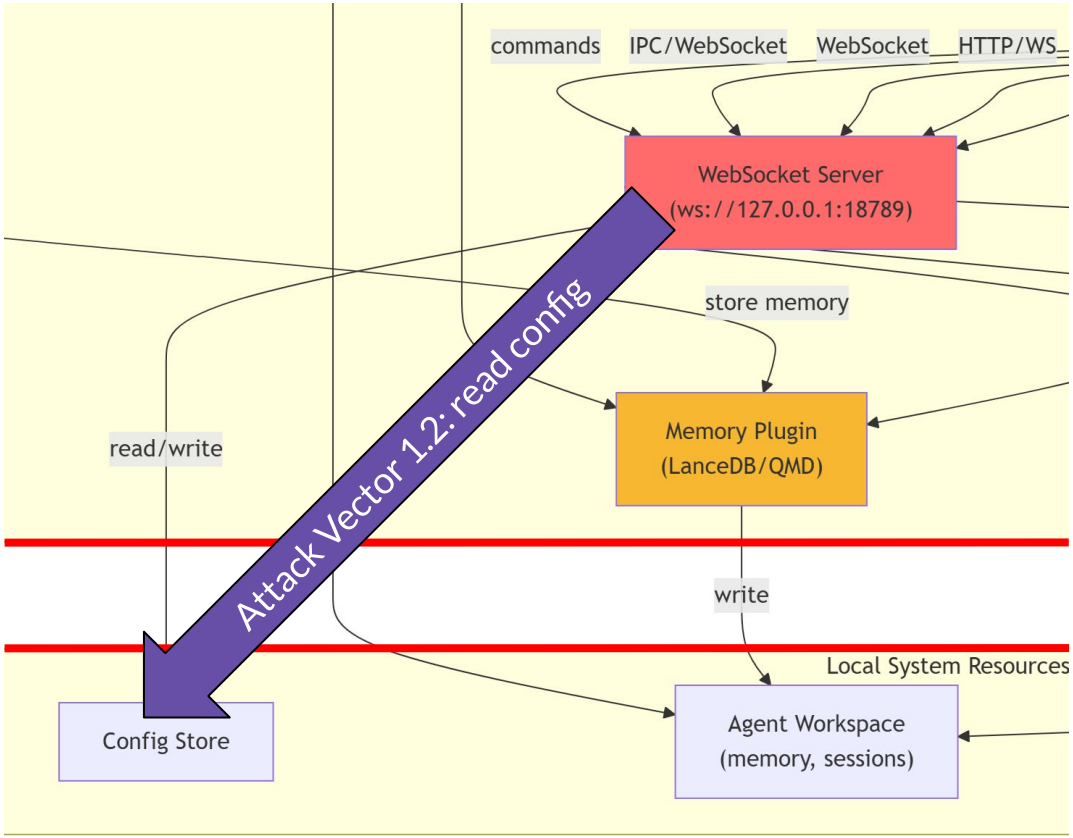
Cloud infrastructure (internal zone) has components directly exposed on the Internet (external zone).

Local developer machine (internal zone) that can also be a target with indirect attacks such as supply chain (external zone).

LLM diagram: Attack vectors



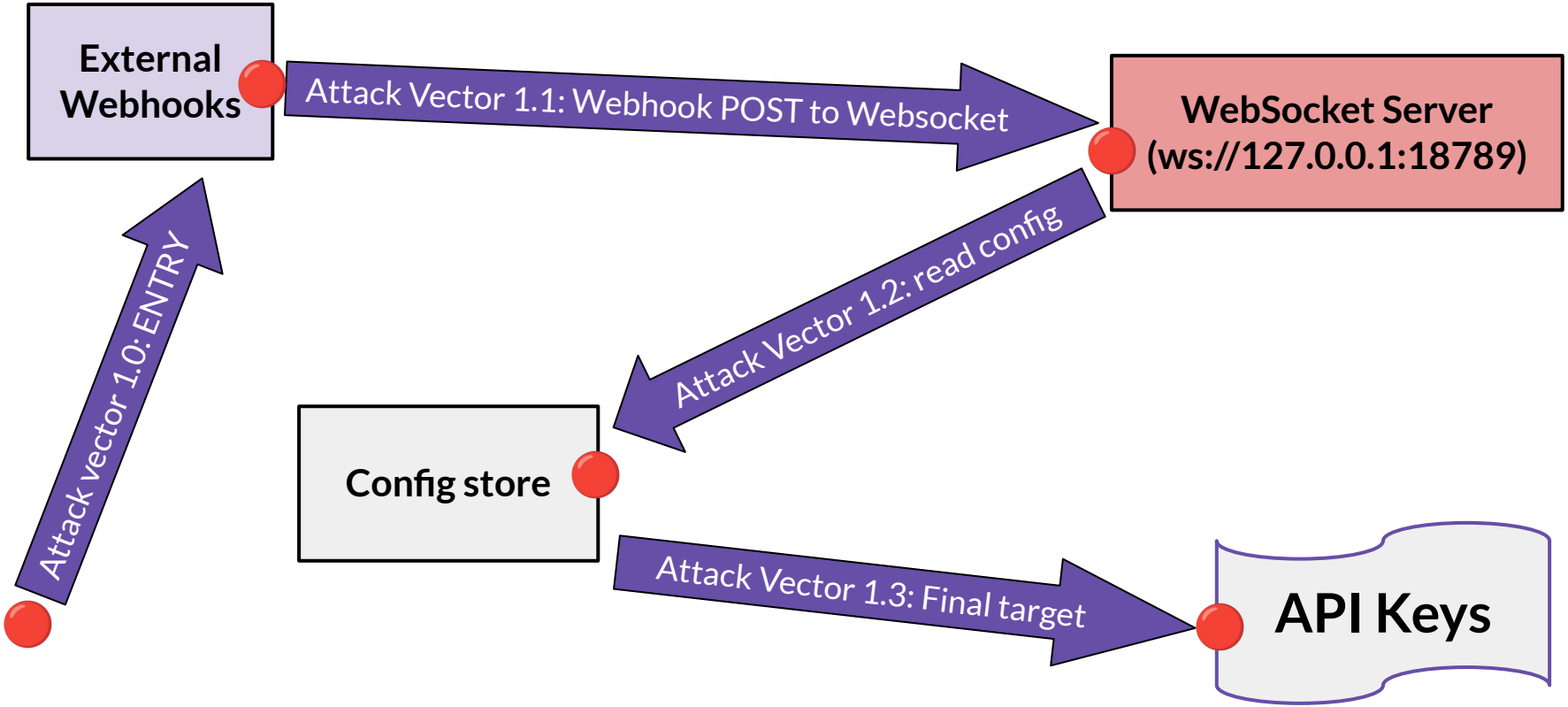
Config Files Attack Vector (cont'd)



Attacker wants your LLM API Keys

So they will use the attack vector to get there...

Attack chain for target API Keys



LLM: What Is Probable to Be Targeted

High-Priority Targets (most likely/impactful)

1. **API Tokens & Credentials:** Discord bot tokens, GitHub tokens, Eve Bubbles passwords in config files
2. **SSH Private Keys:** For remote gateway access
3. **Local Gateway Port:** Unauthorized command execution on local machines
4. **Plugin Ecosystem:** Malicious community plugins or dependency poisoning
5. **LLM Prompt Injection:** Crafted user messages to extract secrets or change behavior

LLM WINS

Have the LLM sort out its own "thoughts"

vibe threat modeling?

LLM: Recommended Mitigations

1. ✅ SSH tunnel for remote access
2. ✅ Memory auto-capture now requires explicit opt-in
3. ✅ Plugin script execution disabled (--ignore-scripts)
4. ⚠️ TODO: Encrypt stored credentials
5. ⚠️ TODO: Use wss:// for all WebSocket connections
6. ⚠️ TODO: Implement local gateway authentication/rate limiting
7. ⚠️ TODO: Sanitize all user inputs before LLM context
8. ⚠️ TODO: Add plugin code review/signing mechanism
9. ⚠️ TODO: Implement comprehensive audit logging

What is.. Data Handling?

Systems and humans handle data, instead of a software lifecycle we talk about a data lifecycle

Knowing what happens during that lifecycle indicates when to take action to secure data

when to take actions:

Data Lifecycle

- Creation
- Distribution
- Implementation
- Usage
- Update
- Decommission

Pause to assimilate big picture

And maybe take a sip of water?

Let’s get into action

Practical actions to take during Data Lifecycle…

LLM API Keys

LLM API Keys

Creation

Developer create API key at the LLM provider level or cloud platform.

A web interface is used, but it could be programmatic with APIs and terraform.

The key is linked to a billable or quota restricted account.

Restrict access

Use security group to ensure selected developers can create API Keys.

E.g.: Billy from accounting needs access to spreadsheets not LLM keys creation.

Review keys

Is it possible someone created keys that aren't used anywhere?

Delete them!

Restrict billing

Set realistics limits to billing accounts to prevent resource abuses.

Corporate account

Use a business or enterprise account to be able to manage keys at the admin level.

LLM API Keys

Distribution

Developer copies the API key into their local environment.

The application is being bundled with the key and is deployed.

The key is shared with other developers to work on the project.

Maybe don't copy it locally?

Instead of having the key stored locally, you could gain cloud secret manager access and get it there.

Maybe don't bundle it either?

Use a secret manager upon ~~deploy~~ launch to fetch the keys.

Remember the docker report...

Multiple keys

Maybe the creation phase needed to have a key per environment and per developer instead?

← back to Creation

Share without copy paste

Use a web based secret manager or give access to the LLM platform to other developers, don't copy in Slack!

LLM API Keys

Implementation

The application loads the API key from an environment variable.

Methods are created to call the LLM API with key and process response.

Error messages appears when throttling is happening due to quota, and upon other provider mishaps.

Docker Secrets

Official doc warns that environment variables can leak.

But makes it prone to LFI (local file inclusion) from your app. Tradeoff!

Use native code API calls over tool calling

API are not going to have access to local file system, executing a tool might with params.

Never hardcode keys in git

Git is more than code; its commits!

Use gitleaks to scan for secrets.

Check out logs

Error messages could spill the key.

Watch for misuse of LLM calls like quota busts, unusual usage.

Reminder: GIT is not just HEAD

It's ^HEAD and more than the code itself...

- Commit messages
- Commit history
  - Past code that was deleted
  - Still exists even if you delete a branch
- Distributed copies
  - If you delete in git repo, how about clones?

Tool drop: gitleaks

Run gitleaks docker image on your local repo

```
git clone your-repo.git
cd your-repo/
docker run -v $PWD:/path \
ghcr.io/gitleaks/gitleaks:latest git /path -v
```

https://github.com/gitleaks/gitleaks

LLM API Keys

Usage

The API Key is being transmitted to the LLM provider when the application methods are called.

Endpoints can be chosen dynamically for various models or location depending on criteria.

Make sure your methods aren't used as proxy

Can someone use your app to chat unrestricted?

← back to Implement.

Utilize HTTPS

To verify that the app connects to a legit endpoint.

To encrypt data in transit.

Same with wss://

Privacy

Make sure that endpoints are using models and their locations aligned to your privacy policy.

LLM API Keys

Update

Updating API Keys means to rotate them, which is a security action by itself.

Rotate keys

If any keys is leaked, or just normal prevention with key expiration.

← back to Creation

LLM API Keys

Decommission

The application is shutdown or not using a LLM API Key anymore.

Key rotation has been done.

The endpoint has changed and the application needs a different key.

Delete at the provider

Don't just delete your app, also delete the key in the place your created it.

What about **Attacks to steal PII/IP?**

Maybe it's in your **AI Models**

Threats to models are lower odds due to exposure...

AI Models

(heavily abridged version)

AI Models

Creation

Training a model uses resources and time, which makes it an investment.

Model Poisoning

Unchecked source of data for training can have malicious content that can manipulate outputs outside of what was expected.

Memorization of PII

Make sure that if you train on personal data, there's an anonymisation layer before doing the training so that there's no chances of memorization.

AI Models

Distribution

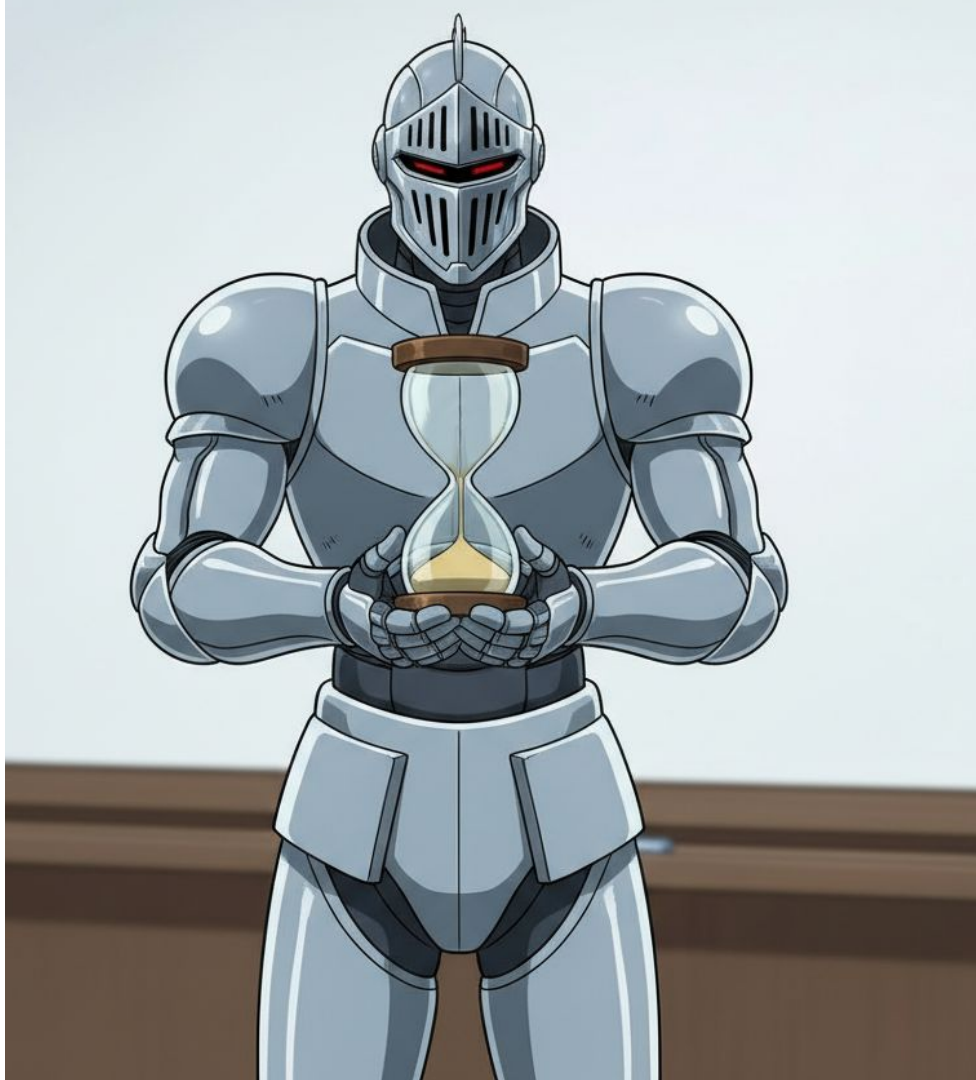
The model is saved into a cloud storage bucket.

World readable open bucket

The cloud storage is misconfigured and its content can be stolen without credentials.

Times up!

Sorry...



Conclusion

What other data can we think of...

Other data related to AI

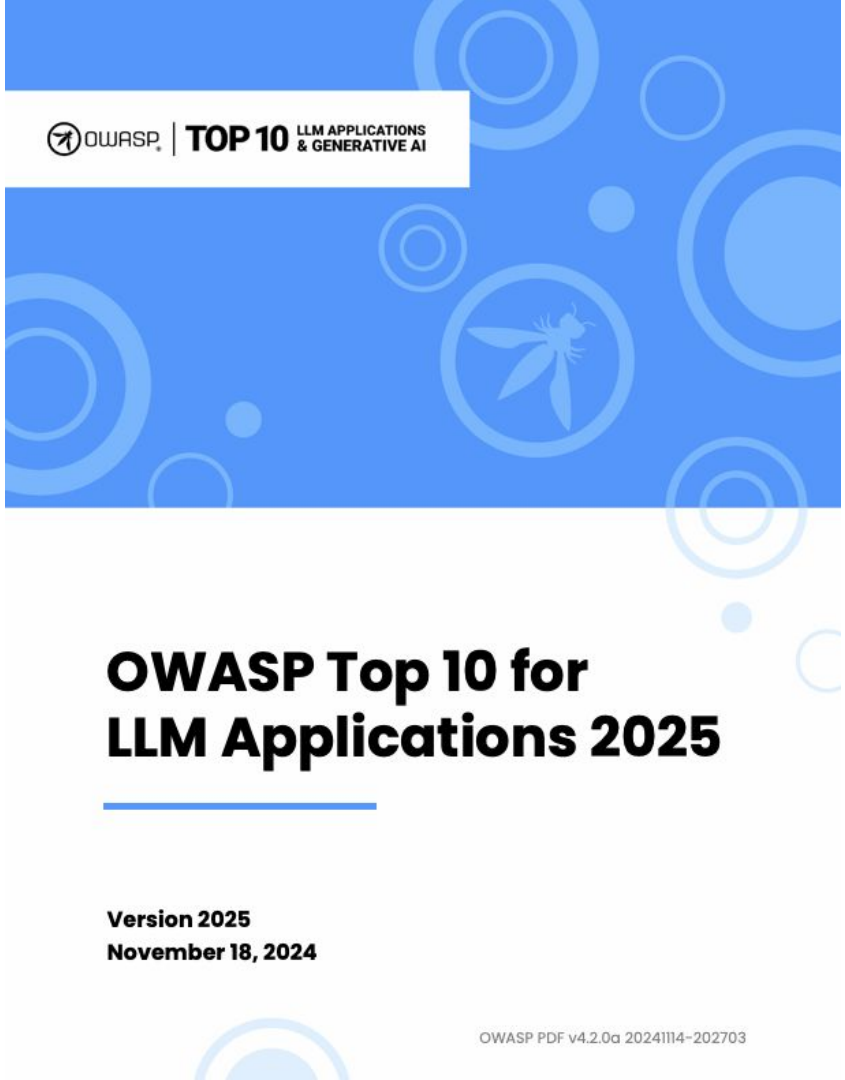
- **System prompts and skills**
  In a total vibe world, these are your source code now.
- **User prompts and chat history**
  While that value is lower than PII, users would still consider it a huge privacy breach if leaked.
- **Source material for training, RAG and CAG**
  If your system uses confidential or proprietary data.
- **Datasets, vectors and embeddings**
  Training and knowledge related processed data that is not the model itself, but contains the results of your efforts and maybe your intellectual property.

Threats are universal

Remember that any AI system is a computer system, which means that any threats could apply to it as well.

Good resource from OWASP to know about security risks related to LLM

Ingest into your favorite LLM context during Threat Modeling!



The news also comes with many new threats we can learn as we go

Stay up to date, your LLM might not be!

Google Cloud Blog

Threat Intelligence

GTIG AI Threat Tracker: Distillation, Experimentation, and (Continued) Integration of AI for Adversarial Use

February 12, 2026

Google Threat Intelligence Group

THANKS!

Slides and links on:

about.jonathanmarcil.ca

Special thanks to:

IVADO Labs

Confoo 2026

Martin

feedback form